



CI/CD

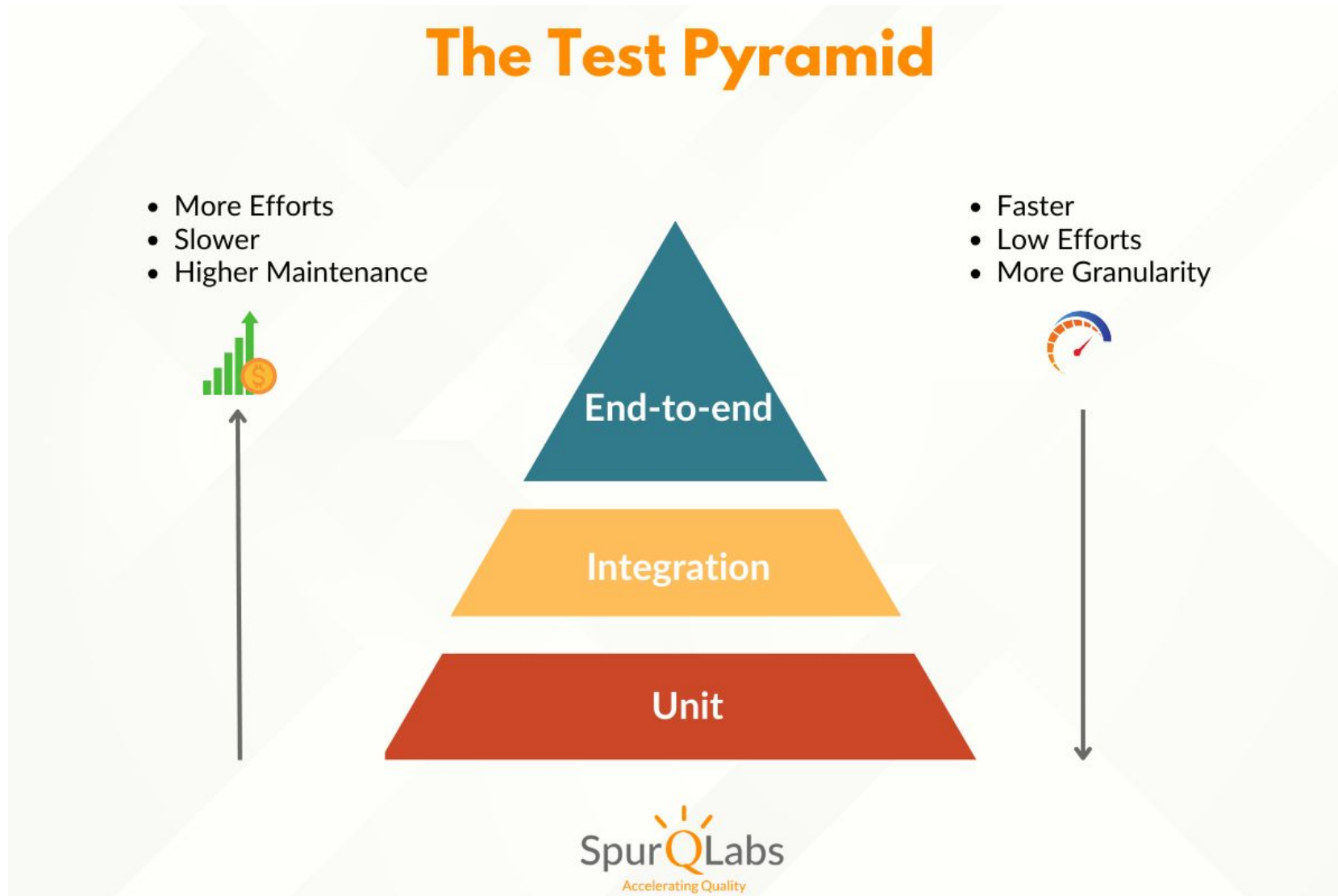
Automating Testing in
CI/CD



Outline

- 1 Unit Testing**
- 2 Integration Testing**
- 3 Security Testing**

Testing Pyramid



1

Unit Testing



Apa itu Unit Testing ?

Unit Testing adalah suatu metode dalam pengujian perangkat lunak yang bertujuan untuk memverifikasi bahwa setiap **unit terkecil dari kode** (biasanya berupa fungsi, prosedur, atau metode dalam sebuah class) berfungsi sebagaimana mestinya secara **terisolasi dari komponen lainnya**.



Penjelasan Detail

Unit dalam konteks ini merujuk pada elemen terkecil dari sistem perangkat lunak yang bisa diuji secara mandiri. Ini bisa berupa:

- ✓ Sebuah fungsi bebas (pure function)
- ✓ Sebuah method dalam class service
- ✓ Sebuah komponen logika bisnis yang tidak tergantung sistem eksternal

Tujuan Unit Testing

Tujuan dari unit test adalah untuk:

- ✓ Memastikan bahwa logika internal dalam unit tersebut benar-benar menghasilkan output yang sesuai dengan ekspektasi berdasarkan input yang diberikan.
- ✓ Menjaga keandalan dan kestabilan kode, terutama ketika terjadi perubahan atau refactoring di masa depan.
- ✓ Unit test ditulis oleh developer saat atau setelah pengembangan kode, bukan oleh QA.

Ciri-Ciri Unit Testing

1. Isolasi:

- Unit test tidak boleh tergantung pada sistem eksternal seperti database, file system, network API, atau framework UI.
- Dependency tersebut biasanya digantikan oleh **mock** atau **stub**.

2. Repeatable & Konsisten:

Harus memberikan hasil yang sama setiap kali dijalankan, selama kodenya tidak berubah.

3. Cepat dan Ringan:

Karena hanya menguji potongan kecil kode, unit test idealnya bisa dijalankan dalam hitungan milidetik atau detik.

Ciri-Ciri Unit Testing

4. Automated:

- Unit test dapat dijalankan secara otomatis dalam pipeline CI/CD, tanpa intervensi manual.

Contoh Scenario

misalkan kita sedang membuat fitur perhitungan diskon harga:



```
public double hitungDiskon(double harga, double persen) {  
    return harga - (harga * persen / 100);  
}
```

Contoh Scenario



Dengan unit test, kamu bisa memverifikasi berbagai skenario:

- `hitungDiskon(100, 10)` harus mengembalikan 90
- `hitungDiskon(200, 0)` harus mengembalikan 200
- `hitungDiskon(0, 50)` harus mengembalikan 0

Unit test akan memastikan fungsi tersebut selalu valid, dan kamu bisa percaya diri melakukan refactor karena hasilnya akan langsung diketahui jika terjadi kesalahan.

Mengapa Unit Testing itu Penting

Manfaat

 Deteksi bug lebih awal

 Mendukung Refactoring

 Otomatisasi

 Dokumentasi Code

Penjelasan

Cegah error sejak dini sebelum masuk integrasi atau produksi

Test menjadi penjamin bahwa refactor tidak merusak fungsionalitas

Bisa dijalankan berkali-kali dalam CI/CD

Test menggambarkan bagaimana fungsi seharusnya bekerja

Tools Unit Test

Bahasa	Framework	Mocking
Java	JUnit, TestNG	Mockito
JavaScript	Jest, Mocha	Sinon
Python	unittest, PyTest	unittest.mock
Go	testing	testify/mock
C#	xUnit, NUnit	Moq

2

Integration Testing



Apa itu Integration Testing ?

Integration Testing adalah proses pengujian perangkat lunak yang dilakukan untuk memverifikasi bahwa modul-modul atau komponen-komponen terpisah dalam suatu sistem dapat bekerja sama secara benar ketika diintegrasikan menjadi satu kesatuan. Tujuan utama dari integration testing adalah untuk menguji titik-titik interaksi antara berbagai bagian sistem dan memastikan bahwa data mengalir dengan benar dan logika bisnis dijalankan dengan tepat pada skala lebih besar dari sekadar unit/fungsi individu.

Mengapa Integration Testing Dibutuhkan ?



Dalam praktik pengembangan perangkat lunak modern, sistem dibangun dari banyak komponen modular:

- Service layer
- Data Access layer
- REST API client
- Authentication provider
- External APIs (payment, shipping, login)
- Caching layer (Redis)
- Messaging system (Kafka, RabbitMQ)

Meskipun setiap komponen ini bisa lulus pengujian unit secara individu, tidak ada jaminan bahwa mereka akan bekerja dengan benar saat digabung. Integration testing berfungsi sebagai penjaga validasi sistem, yang menghubungkan bagian-bagian tersebut dan menguji apakah sistem berjalan dengan benar secara end-to-end—tanpa perlu menjalankan UI atau frontend.

Perbandingan dengan Unit Testing

Aspek	Unit Testing	Integration Testing
Fokus	Satu unit/fungsi	Integrasi antar modul (Service ↔ Repo, Controller ↔ DB)
Ketergantungan	Tidak ada (mock)	Ada (database, API, jaringan)
Kecepatan	Sangat cepat	Lebih lambat
Tujuan	Validasi logika mikro	Validasi kerja sama antar komponen

Pendekatan Integration Testing

Pendekatan Integration Testing

Terdapat beberapa pendekatan umum dalam melakukan integration testing:

Pendekatan	Deskripsi
Top-down	Menguji modul atas terlebih dahulu, modul bawah dimock
Bottom-up	Mulai dari komponen bawah seperti repository, naik ke atas
Big Bang	Menggabungkan semua modul sekaligus lalu diuji
Incremental	Menggabungkan modul satu per satu dan menguji secara bertahap

Integration Testing VS E2E Testing

Aspek	Integration Test	E2E Test
Lingkup	Modul backend saling terhubung	Sistem lengkap dari UI ke DB
Performa	Lebih cepat	Lebih lambat
Ketergantungan UI	Tidak ada	Ya
Tujuan	Validasi kerjasama antar modul	Validasi seluruh alur aplikasi

Kapan Waktu yang Tepat Melakukan Integration Testing?

Kapan Waktu yang Tepat Melakukan Integration Testing?

- Setelah unit testing selesai
- Setelah dua modul saling tergantung sudah siap
- Saat Anda menambahkan alur baru (checkout, verifikasi email)
- Sebelum menggabungkan fitur ke cabang utama (**main/develop**)
- Sebagai bagian dari pipeline CI/CD

Risiko Jika Tidak Melakukan Integration Testing

Risiko Jika Tidak Melakukan Integration Testing

- **Silent failure:** kode tampak “lulus” unit test tapi gagal saat digunakan bersama komponen lain
- **Data corruption:** jika service menulis ke database tanpa integrasi validasi
- **Runtime errors:** kesalahan serialization, dependency injection, incompatibility antar modul
- **Reputasi buruk:** error di produksi akibat kurang validasi integrasi

Komponen yang Sering Diuji dalam Integration Testing

Komponen yang Sering Diuji dalam Integration Testing

- Repository + database (relasional atau NoSQL)
- Service + repository
- Controller + service
- Interaksi dengan sistem eksternal (mock API, OAuth, payment gateway)
- Queue processing (konsumen Kafka/RabbitMQ)

2

Security Testing



2

Hands On



Sebuah aplikasi berbasis java spring boot sedang dikembangkan oleh team development, mereka ingin melakukan otomasi pada CI/CD guna untuk melakukan hal yang berulang, buatkan CI/CD Automation pada Jenkins dengan step berikut

- Git Push ke Branch
- Trigger Jenkins Pipeline menggunakan Webhook
- Menjalankan Unit Testing dan upload ke Sonar
- Menjalankan Sast dan upload ke Sonar

Persiapan:

- Github Repository
- Docker
- Docker Compose

Docker Compose Services:

- **Jenkins** -> akan kita gunakan sebagai platform CI/CD
- **SonarQube** -> akan kita gunakan sebagai static code analysis
- **Cloudflared** -> akan kita gunakan sebagai local tunnel agar Jenkins bisa di hit dari internet agar bisa di integrasikan dengan Github Webhook
- **PostgreSQL** -> akan kita gunakan sebagai database untuk SonarQube

Hands On



```
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
    privileged: true
    networks:
      - cicd-net

  cloudflared:
    image: cloudflare/cloudflared:latest
    container_name: cloudflared
    command: tunnel --url http://jenkins:8080
    networks:
      - cicd-net
    depends_on:
      - jenkins

  sonarqube:
    image: sonarqube:community
    container_name: sonarqube
    depends_on:
      - db
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    ports:
      - "9001:9000"
    volumes:
      - sonarqube_conf:/opt/sonarqube/conf
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_logs:/opt/sonarqube/logs
      - sonarqube_extensions:/opt/sonarqube/extensions
    networks:
      - cicd-net

  db:
    image: postgres:13
    container_name: sonar-postgres
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonar
    volumes:
      - postgresql:/var/lib/postgresql
      - postgresql_data:/var/lib/postgresql/data
    networks:
      - cicd-net

volumes:
  jenkins_home:
  sonarqube_conf:
  sonarqube_data:
  sonarqube_logs:
  sonarqube_extensions:
  postgresql:
  postgresql_data:

networks:
  cicd-net:
    driver: bridge
```

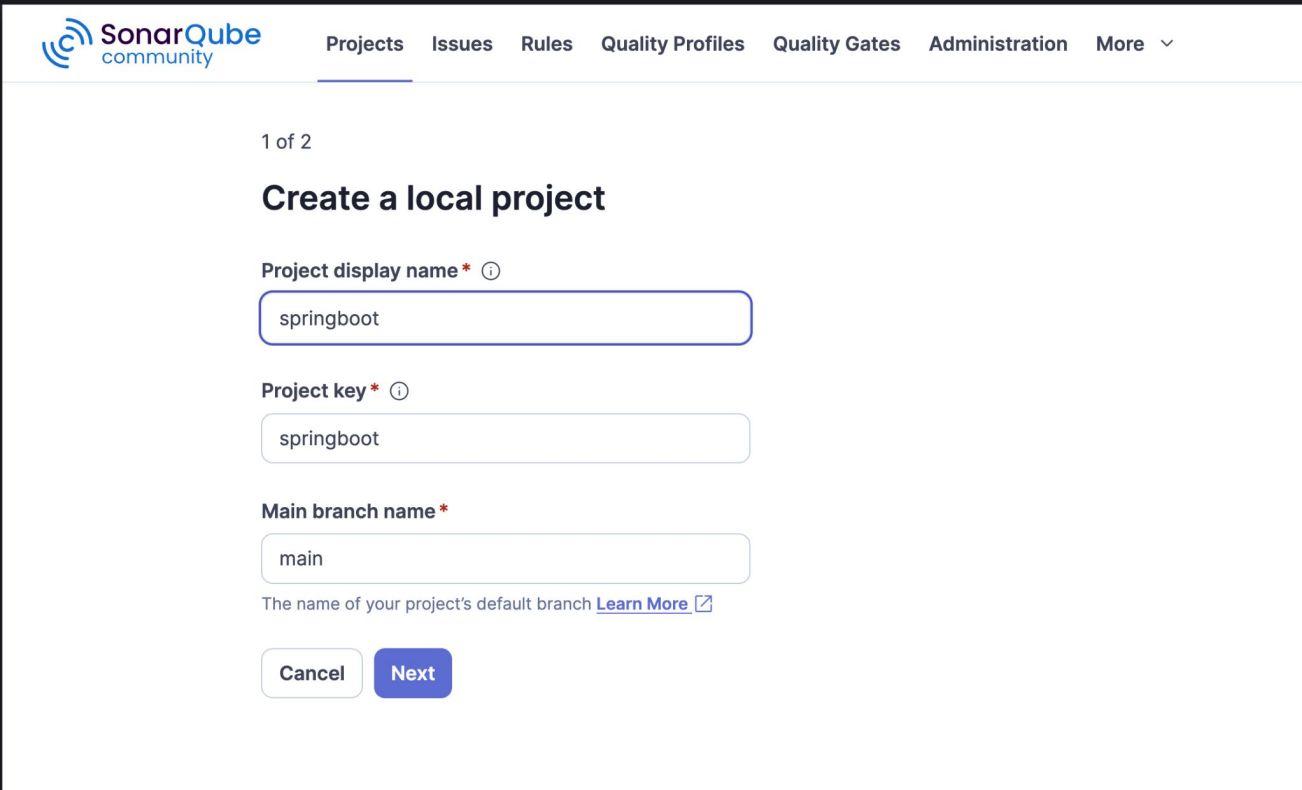
<https://pastebin.com/XZeSYPyL>

Selanjutnya kita akan gunakan sample repo dari Github berikut <https://github.com/noghanodedra/spring-boot-rest-api-unit-tests/> didalam project tersebut merupakan sebuah project spring boot beserta dengan Unit Test dan Jacoco reporting

Silahkan di clone repo tersebut dan push ke github masing-masing, disarankan clone & push bukan fork ya.

Hands On

Silahkan akses port SonarQube dan selesaikan registrasi, kemudian kamu bisa Create a Local Project seperti berikut.



The screenshot shows the SonarQube community interface for creating a local project. The top navigation bar includes links for Projects, Issues, Rules, Quality Profiles, Quality Gates, Administration, and More. The main content area is titled '1 of 2' and 'Create a local project'. It contains three input fields: 'Project display name' with the value 'springboot', 'Project key' with the value 'springboot', and 'Main branch name' with the value 'main'. A note below the last field states 'The name of your project's default branch' with a 'Learn More' link. At the bottom are 'Cancel' and 'Next' buttons.

SonarQube community

Projects Issues Rules Quality Profiles Quality Gates Administration More ▾

1 of 2

Create a local project

Project display name * ⓘ

springboot

Project key * ⓘ

springboot

Main branch name *

main

The name of your project's default branch [Learn More](#) ⓘ

Cancel Next

Hands On



Silahkan membuat access token baru untuk dapat bisa di gunakan dari Jenkins,

Analysis Method / Locally

Analyze your project

We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!

1 Provide a token

Generate a project token

Use existing token

Token name * ⓘ

Analyze "springboot"

Expires in

No expiration



Generate

ⓘ Please note that this token will only allow you to analyze the current project. If you want to use the same token to analyze multiple projects, you need to generate a global token in your [user account](#) [↗](#). See the [documentation](#) for more information.

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your [user account](#) [↗](#).

Hands On

pada Jenkins, silahkan buat 1 Project/Item Baru dengan jenis Pipeline, kemudian kamu bisa menggunakan Jenkinsfile atau groovy script dari sini

<https://pastebin.com/YaJWCRQq>

New Item

Enter an item name



Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.



Organization Folder

Creates a set of multibranch project subfolders by scanning for repositories.

OK

Hands On



Pada Jenkins silahkan coba trigger build

Status

</>

 Changes

📄

 Console Output

✏️

 Edit Build Information

🗑️

 Delete build '#15'

🕒

 Timings

📦

 Git Build Data

📄

 Test Result

🔗

 Pipeline Overview

🔄

 Restart from Stage

🔄

 Replay

📋

 Pipeline Steps

📁

 Workspaces

←

 Previous Build

✅ #15 (Jul 6, 2025, 7:34:41 PM)

🕒

 Started by user [admin](#)

🕒

 This run spent:

- 7 ms waiting;
- 14 sec build duration;
- 14 sec total from scheduled to completion.

📦 git

Revision: 2d975cbf200172c29fd7b1195dd8c5d16101345c
Repository: <https://github.com/noghanodedra/spring-boot-rest-api-unit-tests>

- refs/remotes/origin/master

📄

[Test Result](#) (no failures)

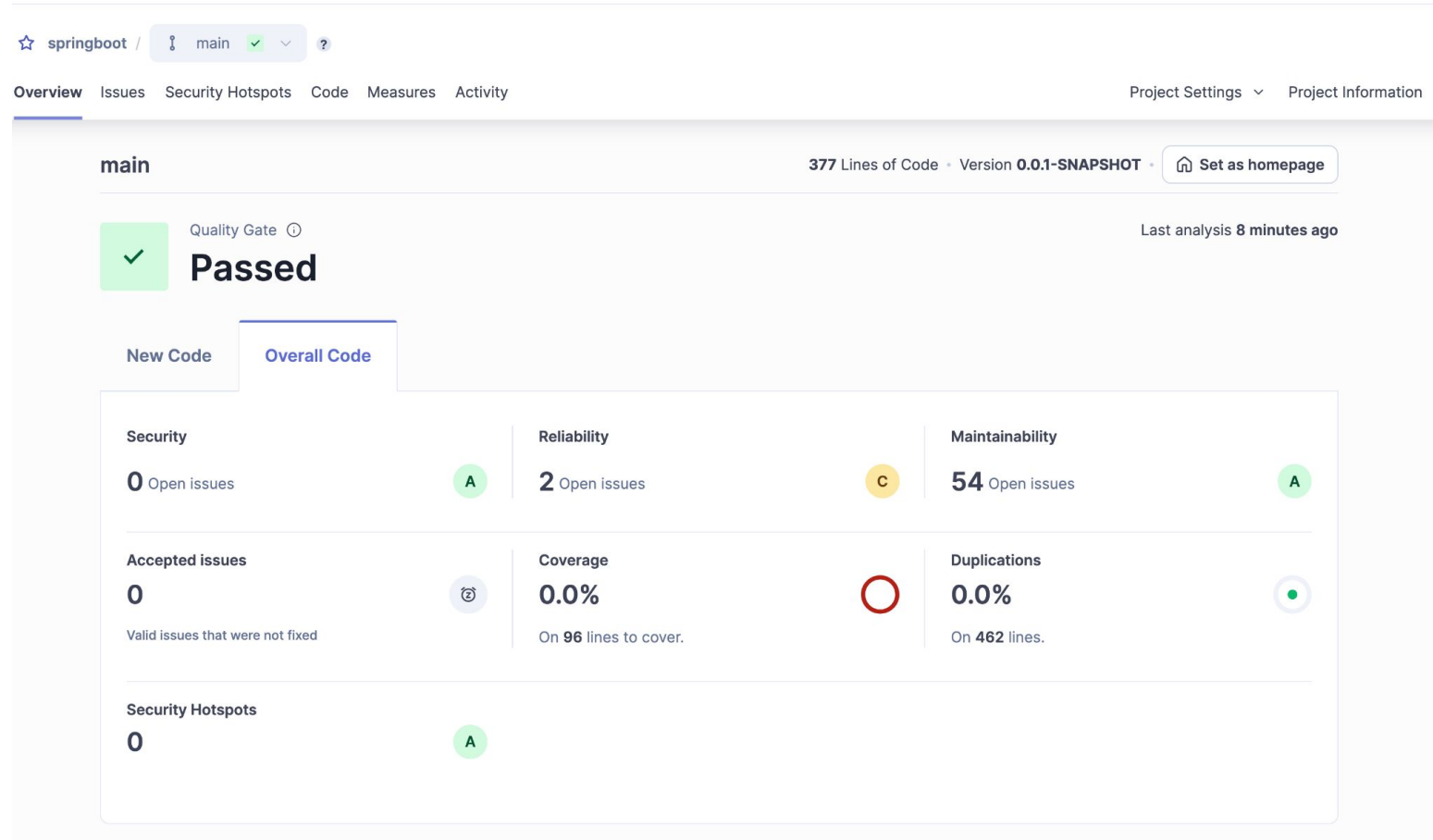
</>

 No changes.

Hands On



Apabila Pipeline telah berhasil
maka pada dashboard
SonarQube akan muncul project
kita



Kamu juga bisa mengintegrasikan webhook untuk melakukan menjalankan pipeline otomatis ketika terjadi push pada repository kamu, caranya pergi ke setting repository kemudian, edit webhook dan masukan http://{subdomain_cloudflared}/github-webhook.

silahkan subdomain_cloudflared bisa diganti dengan subdomain asli yang didapat, kamu bisa cek log container cloudflared

```
jenkins | 2025-07-06 18:31:54Z [id=40] INFO Hudson: Lifecycle:LifecycleReady: Jenkins is fully up and running
cloudflared | 2025-07-06T18:31:54Z INF +-----+
cloudflared | 2025-07-06T18:31:54Z INF | Your quick Tunnel has been created! Visit it at (it may take some time to be reachable): |
cloudflared | 2025-07-06T18:31:54Z INF | https://san-niagara-translated-sense.trycloudflare.com |
cloudflared | 2025-07-06T18:31:54Z INF +-----+
cloudflared | 2025-07-06T18:31:54Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [ ~/.cloudflared ~/.cloudflare-warp ~/cloudflare-warp /etc/cloudflared /usr/local/etc/cloudflared ]
```